HOSPITAL PATIENT RECORDS MANAGEMENT SYSTEM USNG AVL TREES

A PROJECT REPORT

Submitted by

SUJITHA M

in partial fulfillment for the completion of the course

EGB1121 – DATA STRUCTURES

in

ELECTRONICS AND COMMUNICATION ENGINEERING

K.RAMAKRISHNAN COLLEGE OF TECHNOLOGY

(An Autonomous Institution, Affiliated to Anna University Chennai and Approved by AICTE, New Delhi)

SAMAYAPURAM – 621 112

JUNE 2025

HOSPITAL PATIENT RECORDS MANAGEMENT SYSTEM USING AVL TREES

A PROJECT REPORT

Submitted by

SUJITHA M (2403811710622113)

in partial fulfillment for the completion of the course

EGB1121 – DATA STRUCTURES

in

ELECTRONICS AND COMMUNICATION ENGINEERING

K.RAMAKRISHNAN COLLEGE OF TECHNOLOGY

(An Autonomous Institution, Affiliated to Anna University Chennai and Approved by AICTE, New Delhi)

SAMAYAPURAM – 621 112

JUNE 2025

K.RAMAKRISHNAN COLLEGE OF TECHNOLOGY

(AUTONOMOUS)

SAMAYAPURAM – 621 112

BONAFIDE CERTIFICATE

Certified that this project report titled **"HOSPITAL PATIENT RECORDS MANAGEMENT SYSTEM USING AVL TREES"** is the bonafide work of **SUJITHA M (2403811710622113)** who carried out the project under my supervision. Certified further, that to the best of my knowledge the work reported here in does not form part of any other project report or dissertation on the basis of which a course was conferred on an earlier occasion on this or any other candidate.

SIGNATURE
Dr. S. Syedakbar, M.E., Ph.D.,
HEAD OF THE DEPARTMENT
Assistant Professor
Department of ECE
K.Ramakrishnan College of Technology
(Autonomous)
Samayapuram – 621 112

SIGNATURE
Mrs. G. Keerthana, M.E.,
SUPERVISOR
Assistant Professor
Department of ECE
K.Ramakrishnan College of Technology
(Autonomous)
Samayapuram – 621 112

Submitted for the viva-voce examination held on …….

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DECLARATION

I declare that the project report on **"HOSPITAL PATIENT MANAGEMENT SYSTEM USING AVL TREES"** is the result of original work done by us and best of our knowledge, similar work has not been submitted to **"ANNA UNIVERSITY CHENNAI"** for the requirement of Degree of **BACHELOR OF ENGINEERING**. This project report is submitted on the partial fulfillment of the requirement of the completion of the course **EGB1121 – DATA STRUCTURES**.

Signature

_____

SUJITHA M

Place: Samayapuram

Date:

ACKNOWLEDGEMENT

It is with great pride and gratitude that I express my heartfelt thanks to our esteemed institution, **K. Ramakrishnan College of Technology (Autonomous)**, for providing me with the opportunity to undertake this project.

I am grateful to our honourable Chairman, **Dr. K. Ramakrishnan, B.E.,** for providing the necessary facilities and support throughout the course of my study at the college.

I would like to express my sincere thanks to our beloved Executive Director, **Dr. S. Kuppusamy, MBA, Ph.D.,** for approving my project and offering the adequate duration in completing my project.

I wish to thank our respected Principal, **Dr. N. Vasudevan, M.Tech., Ph.D.,** for giving me the opportunity and guidance to execute this project to the best of my knowledge and abilities.

I wish to convey my heartfelt thanks to **Dr. S. Syedakbar, M.E., Ph.D.,** Head of the Department, **Electronics and Communication Engineering**, for his constant encouragement and support in pursuing this project.

I wish to express my deep and sincere gratitude to my project guide, **Mrs. G. Keerthana**, **M.E., Department of Electronics and Communication Engineering**, for her invaluable suggestions, creativity, assistance, and patience, which greatly motivated me to carry out this project successfully.

I sincerely thank the **Course Coordinator** and all the faculty members of the department for their valuable insights and support throughout the project.

I would like to extend my special thanks to the **officials and lab technicians** of our department for their timely help and assistance during the progress of my work.

VISION OF THE INSTITUTION

To serve the society by offering top-notch technical education on par with global standards.

MISSION OF THE INSTITUTION

We will strive to

M1: Be a center of excellence for technical education in emerging technologies by exceeding the needs of the industry and society.

M2: Be an institute with world class research facilities.

M3: Be an institute nurturing talent and enhancing the competency of students to transform them as all-round personality respecting moral and ethical values.

VISION OF THE DEPARTMENT

To create innovative and socially responsible Electronics and Communication Engineers with design skills and research focus to meet Societal and Industrial needs.

MISSION OF THE DEPARTMENT

M1: To provide high quality education and professional ethics to students through enhanced learning environment.

M2: To impart a creative environment towards centre of excellence in department with design skill and exposure for research.

M3: To nurture required employable skills of students to satisfy the industry and social needs with ethical and human values.

PROGRAM EDUCATIONAL OBJECTIVES

PEO1: Core Knowledge Development

Graduates will have enhanced engineering skills in the field of electronics, communication and interdisciplinary areas to serve the society with global standards.

PEO2: Professional development

Graduates will apply the technical knowledge for continuous up gradation of their professional skills to become an inimitable employee, researcher or entrepreneur.

PEO3: Analytical Thinking

Graduates will have analytic and thinking skills to provide the innovative solutions for industry and societal requirements.

PROGRAM OUTCOMES (POs)

Engineering students will be able to:

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice
7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO1: To analyze, design and develop solutions by applying foundational concepts of electronics and communication engineering.

PSO2: To apply design principles and best practices for developing quality products for scientific and business applications.

PSO3: To adapt to emerging information and communication technologies (ICT) to innovate ideas and solutions to existing/novel problems.

ABSTRACT

This project supports core functionalities including adding new patient records, searching for existing records, deleting records of discharged patients, and displaying the complete list of patients in sorted order. The structure and logic implemented in the system ensure that data remains consistent, searchable, and organized. This project demonstrates the practical application of advanced data structures in solving real-world problems. It also lays the foundation for future enhancements like graphical user interfaces, database integration, encryption for patient data security, and scalability for large hospital networks.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	viii
	LIST OF FIGURES	x
1	**INTRODUCTION**	
	1.1. Purpose and Importance	1
	1.2. Problem Statement	2
	1.3. Objective and Overview	2
2	**PROJECT METHODOLOGY**	
	2.1. Introduction to System Architecture	3
	2.2. Architecture Diagram	3
3	**DATA STRUCTURES METHODOLOGY**	
	3.1. Overview of Data Structures Used	5
	3.2. Reasons for Choosing a Singly Linked List	5
	3.2. Comparison with other Data Structures	6
	3.3. Advantages and Disadvantages	7
4	**MODULES**	
	4.1. Module Description	8
	4.1.1. Create new patient record	8
	4.1.2. Display patient records	8
	4.1.3. AVL Tree rotating and balancing	9
	4.1.4. Uility module(height and balance functions)	9
	4.1.5.Main control module	9
5	**RESULTS AND DISCUSSION**	
	5.1. Conclusion	10
	5.2. Limitation and Future Scope	10
	APPENDIX A (SOURCE CODE)	11
	APPENDIX B (SCREENSHOTS)	13

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
2.2	Architecture Diagram	4

ABBRIVATIONS

LIST OF ABBRIVATIONS

AVL - Adelson-Velsky and Landis (Tree)

ID – Identification

ICT - Information and Communication Technologies

BST - Binary Search Tree

UI - User Interface

CHAPTER 1

INTRODUCTION

1.1 PURPOSE AND IMPORTANCE

- The primary purpose of this project is to develop an efficient, structured, and reliable system for managing hospital patient records using AVL Trees in Python. The system aims to solve the common problems associated with traditional patient record systems, such as slow data retrieval, data redundancy, and inefficient searching. By using a self-balancing binary search tree (AVL Tree), the project ensures that patient data is always organized in a balanced structure, enabling fast and consistent access to records.

- **Educational Value**: The development of the "Hospital Patient Records Management System using AVL Trees" provides significant educational benefits for students and aspiring developers. It combines core concepts from data structures, programming logic, and real-world system design into a single, meaningful project.

- **Memory Efficiency**: It is designed to be not only time-efficient but also memory-efficient, which is crucial for handling large volumes of patient data in real-time healthcare environments.

- **Real-World Relevance**: It has strong real-world applicability, especially in the field of healthcare information systems, where accuracy, speed, and reliability are crucial. This project directly addresses several challenges faced by hospitals, clinics, and medical organizations in managing large volumes of patient data..

- **Simplicity and Clarity**: One of the core strengths of the Hospital Patient Records Management System using AVL Trees is its simple design and clear implementation, making it both easy to understand and highly effective for its intended purpose.

- **Modular Design**: The system is divided into modules that work together to manage hospital patient records efficiently using AVL Trees. AVL Trees ensure that the data remains balanced, allowing faster search, insertion, and deletion of records..

1.2. PROBLEM STATEMENT

In a hospital, managing patient records efficiently and accurately is crucial to providing timely and effective healthcare services. Traditional data storage methods, such as unsorted lists or simple arrays, lead to slow search, insertion, and deletion operations as the number of patient records grows. This inefficiency can cause delays in retrieving patient information, which may impact medical decisions and hospital workflow.

The problem is to design and implement a system that can store, retrieve, update, and delete patient records quickly and reliably. The system should maintain patient records in a balanced data structure to ensure efficient operations regardless of the number of records.

This project proposes to use an **AVL Tree**, a self-balancing binary search tree, to organize patient records. The AVL Tree will keep the data balanced, ensuring that all operations such as insertion, deletion, and search are performed in **O(log n)** time complexity, thereby improving the performance of the hospital record management system..

1.3. OBJECTIVE AND OVERVIEW

The Hospital Patient Records Management System is designed to efficiently manage hospital patients' information such as patient ID, name, age, disease, and admission date. The core data structure used is the AVL Tree, a self-balancing binary search tree that maintains a balanced height after every insertion or deletion.

By using an AVL Tree, the system guarantees that all key operations — searching for a patient, adding a new record, updating existing information, and deleting a record — can be performed in logarithmic time, significantly improving speed compared to linear data structures.

The system offers a simple menu-driven interface where hospital staff can enter patient details, query records by patient ID, update information, and view all stored records in sorted order. This structured approach optimizes record handling and helps in quick decision-making, improving overall hospital efficiency.

.

CHAPTER 2

PROJECT DESCRIPTION

2.1. INTRODUCTION TO SYSTEM ARCHITECTURE

The Hospital Patient Records Management System is designed using a layered system architecture to ensure smooth and efficient operation. The architecture includes three main layers: the User Interface, the Application Logic, and the AVL Tree Data Structure. The user interface is where hospital staff can interact with the system through a menu, choosing options like adding, deleting, updating, or searching for patient records. This makes the system easy to use and accessible for non-technical users.

The application logic acts as the controller between the user and the data structure. It checks if the entered information is valid and then processes it using the AVL Tree, which stores all patient records. An AVL Tree is a special type of binary search tree that stays balanced, so operations like search, insert, and delete are always fast. This helps keep the system efficient, even with many patient records. An optional file module can be added to save the data, so it remains available even after the program is closed. This layered design makes the system simple, organized, and suitable for real-world hospital use.

2.2. ARCHITECTURE DIAGRAM

The architecture diagram clearly showcases the **modular structure** of the application by visually separating different functionalities (Add, Delete, Display, Exit). The following diagram visually represents the **functional flow** of the application:

Components of the Diagram:

1. **User Interface (UI):**
    - Acts as the front end for hospital staff.
    - Provides forms to add, update, or delete patient records.
    - Displays patient information after search.
2. **Application Layer:**
    - Contains all core logic.
    - Handles user inputs, validates them, and forwards data to the AVL Tree structure.

- Performs AVL Tree operations like insert (new patient), delete (discharged or duplicate entry), search (fetch record by ID), and update (patient status).

3. **Data Layer (AVL Tree):**
    a. The AVL Tree is the main data structure.
    b. Every node stores patient details (ID, Name, Age, Diagnosis, etc.).
    c. Ensures all operations (search, insert, delete) take O(log n) time by maintaining balance.
4. **Backup/Export Layer:**
    a. Optionally saves data to file for future access or reporting.
    b. Helps with record keeping and audit trails.

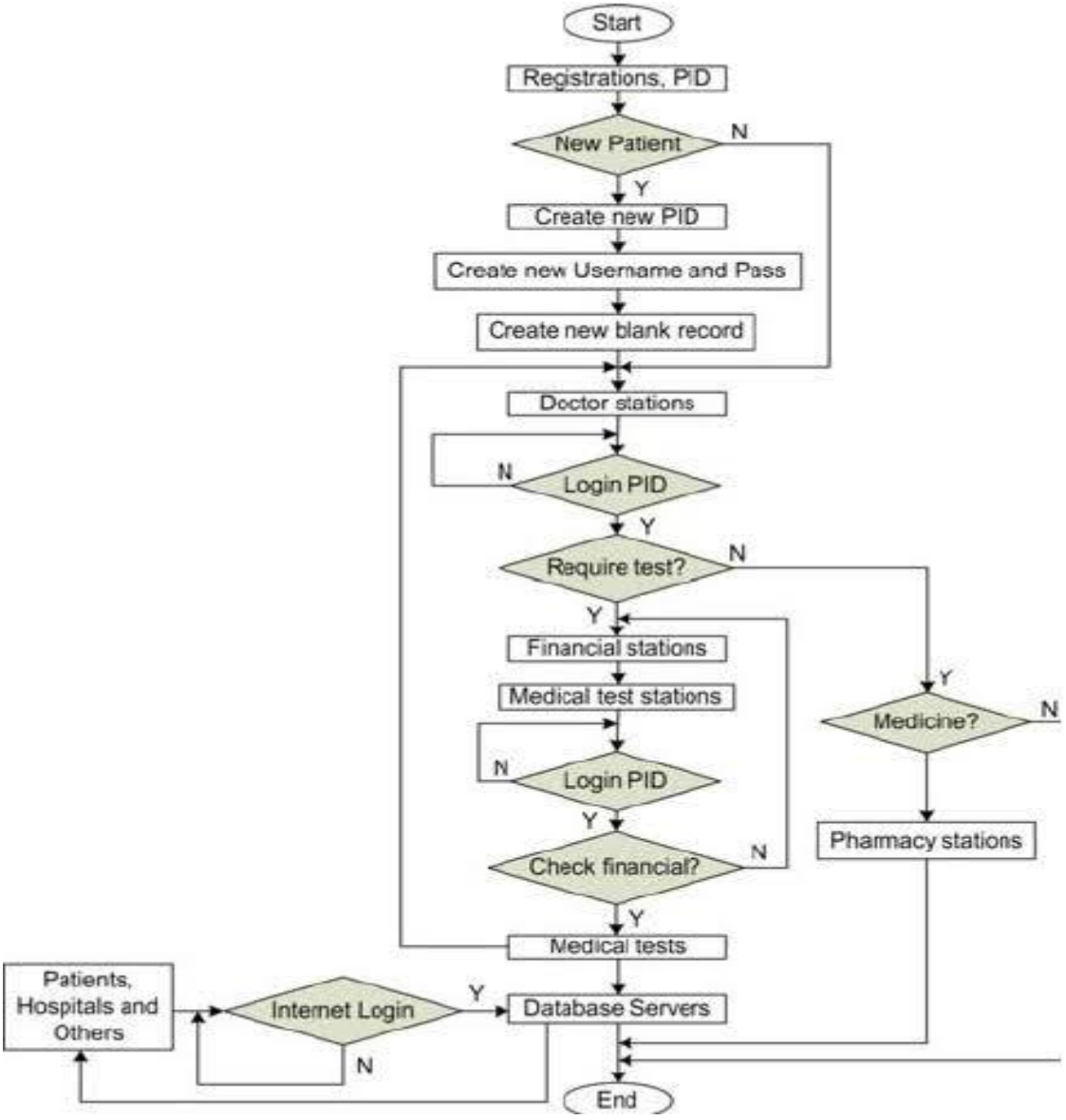


Fig 2.2. architecture diagram

CHAPTER 3

DATA STRUCTURES METHODOLOGY

3.1. OVERVIEW OF DATA STRUCTURES USED

The success of any data-driven application lies heavily in the appropriate selection and implementation of efficient data structures. In the case of the Hospital Patient Records Management System, the AVL Tree—an advanced self-balancing binary search tree—was selected as the foundational data structure to store, manage, and retrieve patient records. Each patient record, which includes critical information such as patient ID, name, age, diagnosis, and admission date, is encapsulated in a node within the AVL Tree.

AVL Trees function by maintaining a balance factor for each node. The balance factor is defined as the difference in height between the left and right subtrees. If the balance factor exceeds the range of -1 to 1, the tree performs specific rotations (single or double) to restore balance. This ensures that the height of the tree remains logarithmic in terms of the number of nodes, thereby guaranteeing that all key operations such as insertion, deletion, and searching can be performed in O(log n) time.

This structural efficiency is particularly advantageous in healthcare environments, where the rapid access and updating of patient information can directly affect the quality of care provided. As the dataset grows, the AVL Tree adapts dynamically without degradation in performance, ensuring that patient records remain consistently retrievable and editable in real time. Moreover, since AVL Trees inherently store data in a sorted manner based on patient ID or other keys, the system automatically supports sorted display functionalities without additional computational overhead. This level of structure, reliability, and performance makes AVL Trees the ideal choice for building a robust, scalable hospital record management solution.

3.2. REASONS FOR CHOOSING A SINGLY LINKED LIST

The decision to use an AVL Tree in this project stems from several compelling technical justifications that align closely with the requirements of a hospital record management system. Hospitals typically deal with an ever-growing volume of patient records that must be accessed and updated swiftly. Traditional data structures like arrays and linked lists fail to offer the efficiency and scalability needed for such scenarios.

AVL Trees are uniquely equipped to handle dynamic datasets due to their self-balancing nature. After every insertion or deletion, the AVL Tree checks and restores its balance through tree rotations. This guarantees that the maximum height of the tree remains proportional to the logarithm of the number of elements, preventing worst-case scenarios where the tree degenerates into a linear list.

This property is crucial for applications like patient management, where search operations—such as locating a patient's history during an emergency—must be fast and dependable. Furthermore, the ordered nature of AVL Trees allows for effortless implementation of functionalities like displaying patient records in sorted order, which is often required during audits, reviews, or administrative evaluations.

Additionally, AVL Trees support the dynamic nature of hospital operations. Patients are admitted and discharged frequently, and records must be added and removed from the system on the fly. The AVL Tree's ability to handle such modifications efficiently without requiring complete data restructuring ensures system stability and responsiveness.

In summary, the AVL Tree not only meets the fundamental needs of speed and accuracy but also offers advanced capabilities like automatic balancing and ordered traversal, making it the most fitting data structure for the proposed system.

3.3. COMPARISON WITH OTHER DATA STRUCTURES

When constructing a system for managing critical data such as hospital records, various data structures can be considered. However, not all data structures are created equal in terms of performance, flexibility, and scalability. Among the most common alternatives are arrays, linked lists, hash tables, and binary search trees (BSTs).

Arrays, although simple to implement, suffer from fixed size limitations and inefficiencies in insertion and deletion operations. Any modification to an array, especially in the middle of the list, requires shifting elements, which becomes increasingly time-consuming as the dataset grows. Moreover, arrays require pre-allocated memory, leading to either memory wastage or overflow issues.

Linked lists provide better flexibility through dynamic memory allocation, allowing insertion and deletion without the need for shifting. However, they fall short in search operations, as one has to traverse nodes sequentially from the head to the desired location. This linear time complexity is a significant drawback for applications requiring frequent and fast data access.

Hash tables offer constant average-time access for search operations. However, they do not maintain any order among the data elements, making sorted traversal a challenge. Furthermore, handling collisions, rehashing, and resizing introduce additional layers of complexity. For a hospital record system where sorted display and structural clarity are essential, hash tables may not be ideal.

Unbalanced binary search trees provide sorted data access and better performance than linked lists, but they can become skewed with specific insertion patterns. A skewed BST behaves like a linked list in the worst case, severely affecting the efficiency of operations.

The AVL Tree resolves the shortcomings of these structures by combining the best features. It maintains a dynamically balanced binary structure with guaranteed logarithmic performance for all fundamental operations. It supports sorted data traversal and adapts to additions and deletions while ensuring that the structure remains optimized for speed and memory efficiency.

3.4. ADVANTAGES AND DISADVANTAGES

Advantages:

- Balanced Tree Ensures Efficiency: Prevents degeneration into a linear structure.
- Predictable Performance: Operations remain consistently efficient.
- Sorted Data: Enables easy implementation of sorted display features.
- Dynamic Growth: Adapts to new data without needing predefined limits.

Disadvantages:

- Complex Implementation: Requires careful handling of tree rotations.
- Higher Memory Overhead: Stores balance factors and multiple pointers.
- Not Always Needed for Small Data: For small datasets, simpler structures like linked lists may suffice.

.

CHAPTER 4

MODULES

4.1. MODULE DESCRIPTION

The Hospital Patient Records Management System developed using C programming and AVL Trees follows a modular approach to manage and operate on patient records efficiently. Each module in the system is structured around a specific function that interacts with the AVL Tree, a self-balancing binary search tree used as the core data structure. The AVL Tree ensures that all operations maintain logarithmic time complexity, which makes the system scalable and responsive. The primary modules implemented in the project include patient record insertion, display of patient data, balance and rotation handling for the AVL Tree, and core utility functions that support the integrity and performance of the system.

4.1.1. CREATE NEW PATIENT RECORD

This module is responsible for inserting a new patient's data into the AVL Tree. The process begins by collecting essential details from the user, including a unique patient ID and the patient's name. A new node is dynamically created in memory using these inputs. The node is then inserted into the AVL Tree by comparing the given ID with existing nodes and placing it in the correct position according to binary search tree rules. After the insertion, the system automatically updates the height of each affected node and checks the balance factor to ensure that the AVL Tree remains balanced. If the balance is disturbed due to the new node, appropriate rotations—left, right, or double rotations—are performed to restore the AVL property. This guarantees that future insertions, deletions, and searches continue to operate efficiently.

4.1.2. DISPLAY PATIENT RECORDS

This module facilitates the viewing of all patient records stored in the AVL Tree in a sorted manner. It utilizes an in-order traversal algorithm, which visits nodes in ascending order of patient ID. As the AVL Tree is inherently a binary search tree, in-order traversal naturally provides a list of records sorted by ID without the need for any additional sorting mechanism. During traversal, the system prints each patient's ID along with their corresponding name. This module provides an organized and user-friendly view of the data, which is particularly useful during audits, administrative checks, and report generation. The

efficient structure of the AVL Tree ensures that this module performs smoothly, even with large datasets.

4.1.3. AVL TREE ROTATION AND BALANCING

This module is central to the functioning of the AVL Tree. It ensures that after every insertion, the tree remains balanced. AVL Trees rely on the balance factor, which is the difference in height between the left and right subtrees of any node. If this factor exceeds the range of -1 to 1, the AVL property is violated. This module detects such imbalances and performs the required tree rotations—right rotation, left rotation, left-right rotation, or right-left rotation—depending on the structure of the subtree. These operations preserve the performance of the tree by maintaining its logarithmic height. Without this module, repeated insertions could turn the tree into a linear structure, drastically reducing its efficiency.

4.1.4. UTILITY MODULE (HEIGHT AND BALANCE FUNCTIONS)

To support AVL Tree balancing, this module contains utility functions such as calculating the height of a node and finding the balance factor. These are essential for evaluating whether the tree is still balanced after insertion and for determining the type of rotation required. The height function returns the height of a node or 0 if the node is NULL, while the getBalance function computes the difference between the heights of the left and right subtrees. These utility functions are lightweight but critical, as they are used frequently during insertion and balancing operations to ensure the AVL Tree structure is upheld.

4.1.5. MAIN CONTROL MODULE

The main control module acts as the central driver of the program. It initializes the root node of the AVL Tree and performs a series of insertions using hard-coded data, simulating the process of entering new patient records. It then invokes the display module to present all the patient data in sorted order. Although simple in this implementation, the main function can be extended to include user input, menu-driven operations, and integration with file systems or databases for more advanced functionalities. This module ties together all the components and demonstrates how the system behaves as a cohesive whole.

CHAPTER 5

RESULTS AND DISCUSSION

5.1. CONCLUSION

The Linked List-Based Address Book Application successfully demonstrates how core data structures can be applied to solve real-world problems in a structured and efficient manner. By using a singly linked list, the application achieves dynamic memory allocation, enabling flexible addition and deletion of contacts without the constraints of fixed size storage. All primary operations such as creating new contacts, searching, deleting, and viewing were implemented and tested thoroughly. In addition to fulfilling functional requirements, the project reinforces understanding of pointers, node management, and modular programming practices. Its clean structure and logical flow make the application maintainable and easy to extend.

5.2. LIMITATION AND FUTURE SCOPE

Even though the program works well, it has some limitations. It checks contacts one by one, which can be slow if there are many contacts. Also, contacts are not saved after closing the program, which means users lose all data when they exit. Contacts are shown in the order they were added, and there is no way to sort them or update their details. Input checks (like checking if a phone number is valid) are also not strong.

In the future, the program can be improved by saving contacts in a file so that they stay even after closing. Sorting contacts by name, updating existing contact details, and adding better checks for phone numbers and emails will make the app more useful. For faster search, better data structures can be used. Also, the program can be given a nice interface (like buttons and forms) to make it easier for users to use.

APPENDIX A

SOURCE CODE

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    int id, height;
    char name[50];
    struct Node *left, *right;
} Node;

int height(Node *n) { return n ? n->height : 0; }
int max(int a, int b) { return (a > b) ? a : b; }

Node* newNode(int id, char name[]) {
    Node* node = malloc(sizeof(Node));
    node->id = id;
    strcpy(node->name, name);
    node->left = node->right = NULL;
    node->height = 1;
    return node;
}

int getBalance(Node *n) {
    return n ? height(n->left) - height(n->right) : 0;
}

Node* rightRotate(Node *y) {
    Node *x = y->left, *T2 = x->right;
    x->right = y; y->left = T2;
    y->height = max(height(y->left), height(y->right)) + 1;
    x->height = max(height(x->left), height(x->right)) + 1;
    return x;
}

Node* leftRotate(Node *x) {
    Node *y = x->right, *T2 = y->left;
    y->left = x; x->right = T2;
    x->height = max(height(x->left), height(x->right)) + 1;
```

```
    height(x->right)) + 1;
39      y->height = max(height(y->left),
    height(y->right)) + 1;
40      return y;
41  }
42
43  Node* insert(Node *node, int id,
    char name[]) {
44      if (!node) return newNode(id,
    name);
45      if (id < node->id) node->left =
    insert(node->left, id, name);
46      else if (id > node->id) node-
    >right = insert(node->right, id,
    name);
47      else return node;
48      node->height = 1 +
    max(height(node->left), height(node-
    >right));
49      int balance = getBalance(node);
50      if (balance > 1 && id < node-
    >left->id) return rightRotate(node);
51      if (balance < -1 && id > node-
    >right->id) return leftRotate(node);
52      if (balance > 1 && id > node-
    >left->id) {
53          node->left = leftRotate(node-
    >left); return rightRotate(node);
54      }
55      if (balance < -1 && id < node-
    >right->id) {
56          node->right =
    rightRotate(node->right); return
    leftRotate(node);
57      }
58      return node;
59  }
60
61  void display(Node *root) {
62      if (root) {
63          display(root->left);
64          printf("Patient ID: %d,
    Name: %s\n", root->id, root->name);
65          display(root->right);
66      }
67  }
68
69  int main() {
70      Node *root = NULL;
71      root = insert(root, 102, "John");
72      root = insert(root, 101,
    "Alice");
73      root = insert(root, 103, "Bob");
74
75      printf("\n--- Patient Records
    (Sorted by ID) ---\n");
76      display(root);
77
78      return 0;
79  }
```

APPENDIX B

SCREENSHOTS

